Machine Learning in Practice Report

Veronica Bolzonella (s1090236), Ana Leopold (s1093729), Luca Maas (s1096050), Marcelo Ribero Cerro (s1100744)

1 Introduction

In this report we present our solution to the Deep Past Challenge which is about the Akkadian translation. Rather than building upon existing public notebooks, we chose to develop our models from scratch in order to gain a better learning experience. Specifically, we trained both a ByT5 and an mBART model, and submitted predictions from each. ByT5 was a common choice among many participants because it works directly on raw bytes rather than on tokens making it attractive for languages such as Akkadian. The choice for mBART was based on the literature that highlights its performance in low-resource languages [1, 2, 3].We concluded the competition with public scores of 29.2 for mBART and 31.0 for ByT5 and private scores of 29.8 and 31.1 respectively. [1]

The codebase for our implementation can be found here: https://github.com/VeronicaBolzonella/AkkadianTranslation-Sclay.

2 Method

Throughout the competition we conducted a series of experiments to improve our chosen translation models. We finetuned the two models concurrently. For each model we introduced a single new feature iteratively, submitted the modified model to the competition, and only maintained features that resulted in an improved score. [2] Moreover, some conclusions for features that resulted in significant performance drops or improvements for one model were extended to the other model to save time and resources.

In the following subsections we delve in detail about our pipeline.

2.1 Data and Preprocessing

To train our models we used three datasets provided by the competition hosts: (1) Training dataset, (2) Dictionary dataset, (3) Onomasticon [3] and (4) a dataset from the Akkademia project [5]. We also used the provided lexicon and onomasticon for data augmentation (see Section 2.4) and the OARE dataset for one alignment technique (see Section 2.2).

All datasets were preprocessed to normalize the data, standardize annotations and clean artifacts (details in Appendix A). We tested three main preprocessing parameters:

- **Normalization vs Diacritics**: The most common public approach is to normalize all characters to their standard letter with corresponding diacritics (e.g., *sz* to *š*). We test both this approach and the reverse normalization logic, changing all diacritics to their normalized form (*š* to *sz*).
- **Separate logograms compounds**: Akkadian transliteration includes logograms which can sometimes be compounded into one term (e.g., *KÙ.BABBAR* is a compound of logograms *KÙ* and *BABBAR*). We test whether the model benefits more from separating these compounds into two words or keeping the compound.
- **Named determinatives**: Akkadian makes use of determinatives that indicate entity types (e.g., d preceding divine actors). We test the possibility of substituting these determinatives with their English meaning (e.g., {d} → God).
- **Hyphenated words**: Akkadian transliterations are annotated by syllabus (e.g *áš-qú-ul*). We tested whether the model benefits from maintaining or removing hyphens.

[1] Note that the private leaderboard shows we dropped by 1000 places relative to the public leaderboard. This difference arises because the public leaderboard includes an earlier public notebook submission, which achieved a score of 34.2 on the public dataset. For the final submission, however, we opted to submit the models we developed from scratch instead.

[2] There is a risk of overfitting to the public leaderboard through this approach, yet due to the limitations of the training dataset we decided that this approach was more robust than relying on our small validation set.

[3] A collection or listing of words especially in a specialized field (as science or commerce)[4]

2.2 Alignment

The training dataset provided in the competition is split at a document level rather than at a sentence level, meaning that one document can be split into multiple data points. Since the test set is provided at the sentence level, we believed aligning the training data the same way would result in an improvement. We tested the models on the default training data and two synthetic datasets.

- **Syntactic matching alignment**: English sentences are split on punctuation, and matched to Akkadian using the OARE alignment set and verified using provided length and first word.
- **Sentence matching and manual alignment**: We identified common sentence patterns that could be used to split documents with regex. Patterns such as *Seal of* and *Witnessed by*. Additionally, we manually aligned sentences that had a length ratio above 2 or below 1/2 between the translation and the transliteration.

Details on the alignment implementation can be found in Appendix B.

2.3 Pretraining

We tested pretraining in two different (non exclusive) forms: self-supervised learning with a denoising objective, including span corruption for the ByT5 model, and translation objective with additional datasets: onomasticon, dictionary and Akkademia.

2.4 Data augmentation

We create additional data by swapping around proper names of people and geographical places in the train dataset. For example, we use the data point (*dumu a-šùr...*, *Aššur's son...*) to create a new data point (*dumu ša-lim-ar-dí ...*, *Šalim-wardī's son*). For more details about the implementation see Appendix C.

2.5 Vocabulary features

This feature was specifically tested for mBART, because of its sentence piece tokenizer compared to the byte pair approach of ByT5. We added the following tokens in successive tests: (1) token to identify Akkadian language (*akk_XX*), (2) special tokens to mark the gap, start and end of the logogram and (3) around 300 additional tokens of most common Akkadian words (see Appendix D for details) Moreover, we tested initializing the weights of the new token *akk_XX* to the ones of the Arabic language token under the assumption that Akkadian language structure is most similar to Arabic than any other language known to mBART.

2.6 Training and inference features

We evaluated several training features. First, dropout was tuned by testing values between 0.05 and 0.45 during finetuning. Second, we compared training with 16-bit and 32-bit precision. We experimented with two loss functions for mBART: (i) standard cross-entropy loss, and (ii) a combined loss defined as the geometric mean of cross-entropy and a BLEU proxy applied at the logit level (see Appendix E for more details). Additionally, we experimented with different maximum length of tokens for ByT5 trying out values 128, 256 and 512 to be able to increase the batch size and speed up training.

We did not extensively tune the learning schedules and learning rate. Instead, we only referred to the health of the training loss curve to choose the learning rate, and we used the same schedule originally used for training the foundational models. We used a linear schedule with warmup for mBART [6] and a fixed learning rate of 0.0001 for ByT5. For the latter, the cosine scheduler was also tested. We trained mBART with early stopping by monitoring the validation loss and geometric mean of chrF++ and BLEU. At inference time we also tested several parameters, such as the number of beams, length penalty and MBR.

2.7 Ensembling

To leverage diversity and strengths of different models we experiment with two ensemble configurations: an ensemble of our two models, and an extended ensemble with our two models and a publicly available ByT5 that scores 35.9 on the public leaderboard.

We performed ensembling using Minimum Bayes Risk (MBR) consensus decoding. For every input, each model generates a set of candidate translations consisting of its most top-ranked beam outputs and two additional stochastic

samples obtained using temperature-based sampling. Since the total number of candidates was constrained by the competition's submission time limits, it was set the maximum feasible number of beams.

For the two-model configuration, each model generated four candidates. For the three-model setting, the public model generated eight beam candidates, while our models contributed four candidates (2 each). This resulted in a diverse pool of candidate translations using different models and generation methods.

The final output was selected using MBR, which chooses the candidate that maximizes the expected similarity to all other candidates in the pool (maximize consensus). Candidate scores were weighted according to their model's standalone performance (ByT5: 1.0, mBART: 0.95, public ByT5: 1.1). Although these weights could have been further optimized, time constraints prevented additional tuning.

3 Results

In this section we present the main results in terms of the geometric mean of BLEU and chrF++ in the public test set. Table 1 should be interpreted as a chronological introduction of features. Features with a positive score change were kept for following tests while features with a negative score change were discarded. Therefore, each score change is calculated on the previous highest score obtained. Feature with a dash are those that were included (P) or discarded (N) after tests on the other model, and therefore were never submitted to the competition. Improvements marked in bold indicate features implemented in the final model.

Table 2 contains the results on the public and private leaderboard of the ensembles.

Feature	**Δ Score**	**Public**	**Δ Score**	**Public**
	ByT5		*mBART*	
Syntactic matching alignment	–	8.6	–	10.2
Hyphens removed	**+4.0**	12.6	P	–
Characters normalization	P	–	**+6.5**	16.7
BLEU proxy loss	N	–	−3.8	12.6
Vocab expansion (akk token)	–	–	**+1.7**	18.1
Separate compounds	−9.0	3.6	N	–
Arabic initialization	–	–	**+0.8**	18.9
Vocab expansion (special)	–	–	−2.0	16.9
Named determinatives	**+13.6**	17.2	−0.4	17.7
Name swapping	**+6.0**	23.2	**+0.9**	21.0
Beams=8	**+1.2**	24.4	–	–
No alignment	**+2.7**	27.1	–	–
Manual alignment	**+1.1**	28.2	**+1.2**	20.1
Akkademia 15k samples	**+3.8**	30.9	–	–
Akkademia 10k samples	−0.1	30.8	–	–
Akkademia full	−0.1	30.8	–	–
Dict + Akkademia	**+0.2**	31.0	–	–
Dictionary pretraining	−0.3	30.7	**+3.4**	24.4
MBR inference	−0.4	30.5	**+1.1**	25.5
Dict. + Akkademia pretraining	−0.1	30.6	**+1.7**	27.2
Span corruption pretraining	−1.5	29.3	N	–
Dropout 0.25	–	–	+2.1	28.2
Dropout 0.35	–	–	**+0.6**	28.8
Dropout 0.45	–	–	−0.5	28.3
Self-supervised pretraining	–	–	−0.1	28.7
Full dataset (no validation)	–	–	+0.9	29.2

Table 1: Iterative experiments for ByT5 and mBART. Features with a dash are those that were included (P) or discarded (N) after tests on the other model, and therefore were never submitted to the competition.

While it is possible that ensembling the public model with our own could mitigate potential overfitting to the public leaderboard, we decided not to submit the ensemble, but instead to submit the individual models we created. After scoring on the initially hidden part of the dataset, the best private scoring mBART and ByT5 models scored 29.8 and 31.1, respectively.

Ensemble	Public Score	Private Score	Conclusion
ByT5 and mBART	29.6	29.9	Does not improve the individual scores.
ByT5, mBART, public ByT5	31.0	31.7	Does not improve the individual scores.

Table 2: Ensemble experiments results.

4 Discussion

Several features showed consistent effects across both models. Character normalization, contrary to the hosts' suggestion to train with diacritics, improved performance for both models. Name swapping was effective for both models, and while the synthetic alignment did not yield useful results, manual alignment lead to significant improvements. For mBART specifically, gains were obtained through Arabic initialization, the addition of the Akkadian language token and, the use of dropout.

Pretraining yielded different results. mBART had significant improvements from pretraining on the dictionary and Akkademia datasets, however, ByT5's performance did not improve with any pretraining strategy. Instead, ByT5 achieved its best performance when finetuning on all data using weighted sampling to preserve the original training distribution, achieving a public score of 31.0. Similarly, mBART achieved its highest public score of 29.2 when trained on the full dataset. Although we expected that the ensembles would improve our results, this was not the case. Better results could potentially be achieved by using a wider range of models.

While our process allowed us to test a wide range of features, it also had several limitations. Firstly, the features were implemented iteratively, and were retained or discarded only based on the previous score, ignoring the possible interactions of different features. This means the improvements are firstly, dependent on the order of tests, which was chosen arbitrarily, and secondly, the final models might not be the best combination of features. Furthermore, in some cases we assumed there would be improvements and implemented two features simultaneously. However, due to computational constraints and daily submission limits, testing the features in a combinatorial manner was not feasible. Similarly, we extended the conclusions of some tests across models, under the assumption that these would not have a different effect for the other architecture. Testing all features would allow for a more reliable conclusion.

We also relied on the public leaderboard score as a metric to evaluate improvement, raising concerns of potential overfitting. However, the private leaderboard results suggest that this was not the case. As mentioned before, though, the limited amount of training data caused the validation test to not be a reliable metric of model performance and generalization. The solution write-ups from top-performing teams also reflect this, with the winner completely discarding the provided dataset and constructing their own, while others focused on generating synthetic data.

Finally, we believe that a major limitation of the competition was the quality of the provided datasets. Inconsistent annotations, typographical errors, poor alignment between transliterations and translation, and the lack of reliable alignment information all contributed to the poor quality of the training set. More importantly, the test distribution appeared to be significantly different than the training distribution, meaning we could not rely on our validation scores. As a result, submissions were our primary means of testing our models' performance.

5 Conclusion

In this report we presented our approach to the Deep Past Challenge, using two translation models, ByT5 and mBART, which were finetuned from their pretrained versions. Instead of using existing public models, we opted to develop our own training pipeline independently, giving us a deeper understanding of machine translation. Through iterative experimentation, we explored a wide range of preprocessing, alignment, augmentation, training, and inference strategies. Our models achieved final private leaderboard scores of 29.8 for mBART and 31.1 for ByT5.

Our findings suggest that strong improvements came from data-related additions to the pipeline, particularly alignment, augmentation, and the inclusion of additional training data. The project also highlights the importance of data quality and coverage in machine translation for low-resource languages.

6 Code Notebooks

We developed our project on the GitHub repository linked in the introduction. We then copied the necessary code in the notebooks for the submissions. We therefore believe it is easier to read our code directly from the repository. We specify at the end of the sections which files to refer to to ease navigation.

- mBART Notebook submission:
  `https://www.kaggle.com/code/verobolzonella/mbart-inference-328522?scriptVersionId=306362350`
- Ensemble submission:
  `https://www.kaggle.com/code/lucamaas/luca-ensemble-leopold-roberoc?scriptVersionId=306375725`
- ByT5 submission:[4]
  `https://www.kaggle.com/code/lucamaas/byt5-inference-final?scriptVersionId=306714108`

File structure for code that is used in the submission notebooks:

- `src/models/mBartFineTuner.py`. Includes the mBART model class
- `src/training/byt5_training.py`. Includes full training pipeline of the ByT5 model.
- `src/dataprocessing/datasets.py`. Includes dataset classes (AkkadianEnglishDataset, AkkadianTranslationDatasetT5).
- `src/dataprocessing/patterns.py` and `src/dataprocessing/processing.py`. Includes code for pre and post processing (TextProcessor class).
- `src/evaluation/kaggle.py`. Includes the KaggleInference class used for inference of mBART.
- `src/evaluation/byt5_inference.py` Includes the byT5Inference class used for inference of Byt5.

7 Author Contributions

- **Veronica Bolzonella**: Preprocessing; Data Augmentation; Bookkeeping of mBART results and WandB; implementation of mBART dropout and learning rate scheduler; BLEU proxy function; contributions in syntactic alignment, manual alignment, mBART debugging, ensembling notebook;
- **Ana Leopold**: Complete ByT5 training pipeline, including a custom dataset, trainer adjusted for weighted sampling and ByT5 span corruption; iterative tests of ByT5 features
- **Luca Maas**: Regex-based alignment, ByT5 Minimum Bayes' Risk (MBR), Implementing batch-level–decoding functions and basic framework for ensembling, factorization of ByT5 inference class, bucket batching, assisting in manual alignment
- **Marcelo Ribero Cerro**: Manual alignment, implementation of regex based alignment, data collection (webscrapers), dataset preparations (onomasticon, dictionary, akkademia), mBART vocabulary expansion, mBART Arabic initialization, mBART dropout, mBART LoRA implementation, BLEU and CHRF+ logging, learning rate scheduler, self-supervised for mBART, constrained decoding, Minimum Bayes Risk (MBR) implementation.

References

[1] Surangika Ranathunga et al. "Neural Machine Translation for Low-resource Languages: A Survey". In: *ACM Comput. Surv.* 55.11 (Feb. 9, 2023), 229:1–229:37. ISSN: 0360-0300. DOI: `10.1145/3567592`. URL: `https://dl.acm.org/doi/10.1145/3567592` (visited on 01/30/2026).

[2] En-Shiun Annie Lee et al. *Pre-Trained Multilingual Sequence-to-Sequence Models: A Hope for Low-Resource Language Translation?* Apr. 30, 2022. DOI: `10.48550/arXiv.2203.08850`. arXiv: `2203.08850 [cs]`. URL: `http://arxiv.org/abs/2203.08850` (visited on 01/30/2026). Pre-published.

[3] Yuqing Tang et al. *Multilingual Translation with Extensible Multilingual Pretraining and Finetuning.* Aug. 2, 2020. DOI: `10.48550/arXiv.2008.00401`. arXiv: `2008.00401 [cs]`. URL: `http://arxiv.org/abs/2008.00401` (visited on 01/31/2026). Pre-published.

[4] Note that this a modularized version of the submitted notebook containing additional functionalities such as bucket batching and MBR.

[4] *Definition of ONOMASTICON*. URL: https://www.merriam-webster.com/dictionary/onomasticon (visited on 03/18/2026).

[5] *Akkademia/README.Md at Master · Gaigutherz/Akkademia*. GitHub. URL: https://github.com/gaigutherz/Akkademia/blob/master/README.md (visited on 03/18/2026).

[6] Yinhan Liu et al. "Multilingual Denoising Pre-training for Neural Machine Translation". In: *Transactions of the Association for Computational Linguistics* 8 (2020). Ed. by Mark Johnson, Brian Roark, and Ani Nenkova, pp. 726–742. DOI: 10.1162/tacl_a_00343. URL: https://aclanthology.org/2020.tacl-1.47/.

A Preprocessing

In this section we describe our fundamental preprocessing (excluding the features that were used in tests). First we normalize all gaps in translation and transliteration by converting occurences of 'PN', [xx...], (x), x, xx, '(n broken lines)', '. . . ' to <gap>. We merge multiple gaps occurring one after the other into one single (e.g., <gap>-<gap> → <gap>). In transliteration we mark logograms with the delimiters <lgg_s> and <lgg_e>. Finally we clean additional annotations such as '?' and '!' and in external datasets '#' and '@'. Similarly, in translation we remove annotations such as '(?)', '(!)', '(fem.)', '(plur.)', '§' etc.

We normalize numbers in translation by converting all numbers to digits instead words (up to 4 decimal places). Months are converted to roman numerals and amounts in shekels are converted to the standard shekels and grains amounts.

We expand words that are abbreviated in some translations ('-gold'→ pašallum gold, -textiles → kutānum textiles, tax→ šadduātum tax,). Some texts also include multiple possible translations (e.g., "she/he"). In these situations we preprocess by always choosing the first option.

Similar steps are also performed in postprocessing of the models translation to guarantee the output is in the expected form.

B Alignment

B.1 Syntactic matching alignment

Before proceeding with manual alignment, we attempted to leverage the `Sentences_Oare_FirstWord_LinNum.csv` dataset, which provides the starting word index and spelling of the first word of each sentence in a tablet's transliteration. For tablets present in both the OARE and training datasets, these indices were used to slice the transliteration and translation into sentence-level segments.

Alignment was accepted only when two conditions were satisfied: The Akkadian word at the provided index matched the expected word in the OARE dataset (verified with character normalization), and the number of English sentences obtained by splitting on end-of-sentence markers matched the number of sentences in the OARE dataset for a specific tablet. When either condition was not met, the original translation and transliteration from the training dataset were retained.

Ultimately, this approach did not yield productive results. The OARE dataset covered only a small fraction of tablets in the training data, and of those overlapping entries only around half produced accepted sentence-level pairs. These results are likely caused by two main factors: the inconsistent formatting of both datasets, which caused punctuation-based sentence splitting to fail unpredictably, and frequent misalignment between the two datasets, where a sample which was complete in one dataset was often incomplete or formatted differently in the other.

B.2 Sentence matching and manual alignment

After noticing that certain sentences appear constantly throughout the dataset, we saw that some of these sentences often mark the beginning or end of sections of text in the data. We decided that these patterns were consistent enough to split a data point into multiple data points with high certainty that information is not fragmented across data points. See Table B.2 for all the expressions we created.

English	**Akkadian**
Seal of –	*KIŠIB –*
Witnessed by –	*IGI –*
Sealed by –	*KIŠIB –*
The Kanesh colony gave us for these proceedings and we have our testimony.	*a-na a-wa-tim a-ni-a-tim kà-ru-um kà-ni-iš i-dí-ni-a-tí-ma ší-bu-tí-ni ni-dí-in*

Table 3: English and Akkadian expressions that we matched with regex

Regex matching became harder than anticipated due to the high variability between data points. Take for example the following sentence:

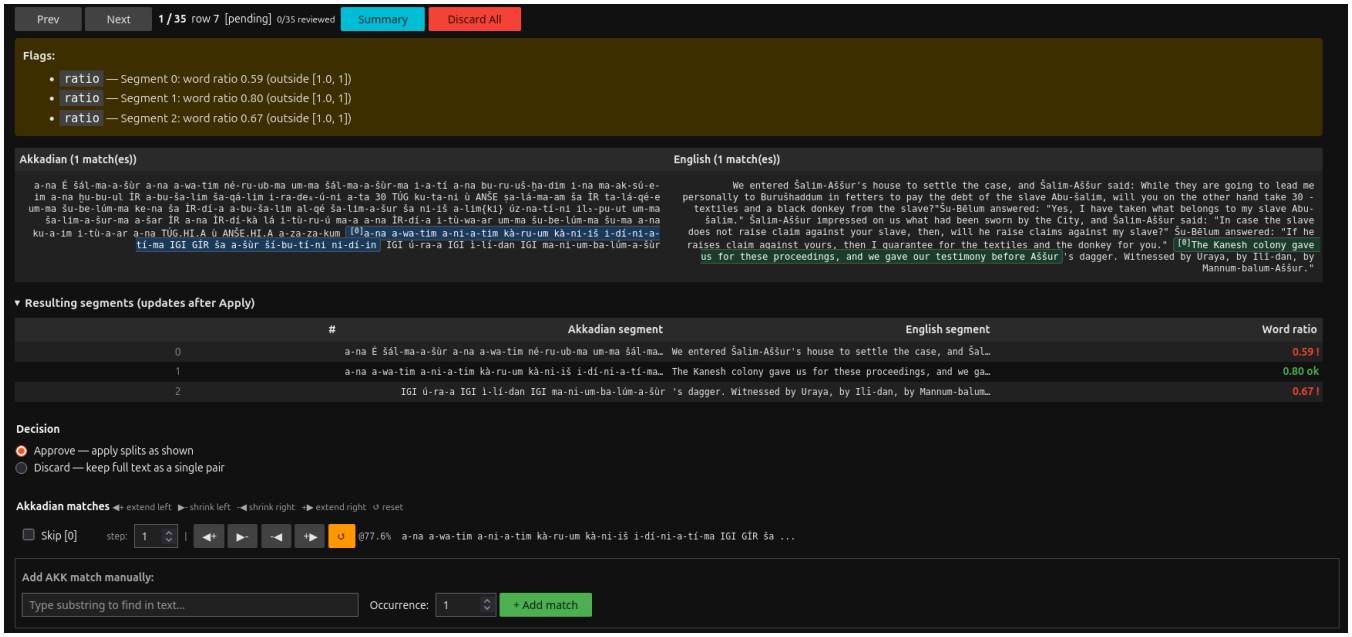

Figure 1: Interactive matching tool. In blue, we have the automatically highlighted Akkadian phrase. On the right side, in green, we have the automatically highlighted English phrase. This can be adjusted with buttons.

IGI ša-lim-a-šur DUMU i-lí-a IGI sá-sí-a
Witnessed by Šalim-Aššur son of Iliya, by Sasiya.

Names of people could often be followed by familiar relations (son, wife, sister) and by professions (Witnessed by Šalim-Aššur, the merchant). Add this to spelling mistakes and different phrasing meant that we needed a human-in-the-loop to verify that we were creating high-quality outputs.

For this, we created an interactive Jupyter notebook in which we could adjust regex matching by inspecting suspicious data points. We had a simple heuristic for detecting suspicious data points, by checking if the word ratio between the Akkadian text and the English text went over a certain threshold (usually 2). After seeing how beneficial this was, we extended this method to all data points, not only the ones matched by our regex expressions. And we divided the dataset into equal parts while we adjusted the data with this interactive tool.

C Augmentation

We perform augmentation for synthesizing data (name swapping), and for creating noisy input pairs (token noise and gap noise).

C.1 Name Swapping

The algorithm searches for proper names of people and geographical sites (e.g., city names). Names of people are found using the onomasticon as a lookup dictionary. Since the translation of names is not always consistent, the english name in the onomasticon sometimes fails to be identified in the translation. In this case the name is found by using the normalized form in the lexicon. If both methods fail, no augmentation is performed. Geographical names are found among the words marked with 'GN' in the lexicon and using the normalized form as the English translation. For both swapping types Akkadian words are stripped of morphemes before being looked up in the onomasticon or lexicon. Once all name pairs in transliteration and translation have been found, a new data point is created by substituting all occurrences of each pair in one data point with a new randomly sampled pair. People names are sampled from the onomasticon, while geographical names are sampled from the lexicon.

C.2 Noise corruption (mBART)

Noise is added to a sentence in two ways. Firstly, noise is added to the token by computing one noising action at either a word level or character level in 20% of the tokens. For each index to be noised, 20% of the times the token gets swapped with the following token, 20% of the time the token gets removed, and the remaining times it remains intact. Secondly, we add gap noise (similar to span corruption), by sampling at random 10% of the tokens and either removing the token (10% of the times), or substituting the token with <gap> (10% of the time). The remaining 90% of times, the tokens remain intact.

C.3 Span corruption (ByT5)

In span corruption random spans of tokens are replaced with unique sentinel tokens. The aim is for the model to learn to reconstruct the missing tokens. This serves as a pretext task where the model is meant to learn the Akkadian features which it can use later in the main translation task. In our implementation, we set the 15% of tokens to be masked with an average span length of three tokens. Ideally, these parameters would be optimized, however, due to a lack of time this was not done which could be the reason why span corruption did not lead to any improvements.

Span corruption is implemented with a custom Data Collator which generates corrupted inputs while the targets are the masked bytes and their corresponding sentinels.

D Vocabulary expansion

This approach intended to explore the impact of language specific tokenization on performance. On average, the amount of tokens per Akkadian word was around 8. We wanted to bring the number down without adding too many tokens. We calculated that 800 extra tokens would be able to get the average token per word down to 3.8, after 800 tokens we started to get diminishing returns. We selected 800 tokens with HuggingFace's SentencePiece Trainer, which learns the most common n-grams, on the Akkadian data we had available. After deduplication with mBART's already existing tokens we ended up adding 300 tokens.

This tokenization meant that we were able to increase the batch size and training speed. Unfortunately, it did not bring any improvement in performance.

We believe that a possible explanation for the lack of improvement is that n-gram tokenization is very sensitive to spelling mistakes and this was common with the data we were provided. Leading to poorly learned tokens.

E BLEU proxy

The score is measured as the geometric mean of BLEU and chrF++. Neither of these measures are differentiable, and therefore it is not possible to optimize for the score directly using backpropagation. We therefore decided to attempt using a differentiable proxy of BLEU that operates at a logit level. One implementation of differentiable BLEU is already available in torch (`bleu_torch.DifferentiableBLEULoss`). However, we were not able to use this implementation, as it creates a VxV matrix (where V is the vocabulary size) which was too large to fit in our GPUs RAM. We designed a lower resource alternative of a differentiable BLEU proxy which utilizes less memory.

First the logits for each token are converted to a log-probability distribution using log-softmax. We then iterate over the sequence tokens for sequences of at least 4 tokens. We extract the log probability of the correct token for each index using the labels, and we assign a geometrically decreasing weight for each token probability depending on their position ($w_t = \frac{0.95^t}{\sum_{j=0}^{T-1} 0.95^j}$). The motivation behind this weight is that BLEU favors shorter translations. We finally calculate the weighted average of the sequence probability and negate the result to obtain a loss:

$$\mathcal{L}_{seq} = -\sum_{t=0}^{T-1} w_t \log p(y_t|\mathbf{z}_t) \tag{1}$$

To not overly rely on this loss we compute the final loss as the mean of cross entropy and BLEU proxy loss. Note that this loosely resembles BLEU as it only computes unigram probabilities, and the initial weight of 0.95 is chosen arbitrarily and not tuned. However, since this loss is slow and did not show promising results, we decided to not explore this direction further.

F Features not submitted

We implemented some additional features that are not mentioned in the report, because we never submitted these models to the competitions. This is because we had reason to believe the model would benefit from these features, but we did not see promising results in training and decided to focus on other directions. Nonetheless, with more extensive work, we do not exclude the possibility that these features could have been beneficial. Below is a list and a brief explanation of these experiments.

- **Constrained decoding**: We implemented a beam selection approach that would prefer sentences that contained non-ambiguous tokens that we expected to see in the generated sentences. For example, if the Akkadian text contained 3 *KÙ.BABBAR* mentions, we would prefer sentences that had *silver* (its translation) 3 times. After a few experiments we realized that implementing this properly would take more time than what we had available so we did not continue with it. Nonetheless, we believed it had potential to improve the sentence selection.
- **mT5**: We briefly experimented with a mT5 model, which did not show promising results in validation and was therefore not experimented further.